



**TECNOLÓGICO
NACIONAL DE MÉXICO**



TECNM - INSTITUTO TECNOLÓGICO DE CULIACÁN

ASIGNATURA:

TÓPICOS DE INTELIGENCIA ARTIFICIAL

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROYECTO:

**IMPLEMENTACIÓN DE ALGORITMO DE ÍNDICE NDVI EN LA DETECCIÓN DE
ESTRÉS HÍDRICO EN CULTIVOS**

NOMBRE DEL MAESTRO:

ZURIEL DATHAN MORA FÉLIX

NOMBRE DEL ESTUDIANTE(S):

GALLEGOS RAMÍREZ HILARIO

RODRÍGUEZ MORENO AMÍLCAR

CULIACÁN SINALOA, 01 DE JUNIO DEL 2026.

1. Índice

1. Índice	1
2. Introducción	2
3. Objetivos de la investigación	2
3.1 Objetivo general	2
3.2 Objetivos específicos	3
4. Justificación	3
5. Alcance del proyecto	4
5.1 Qué incluye	4
5.2 Qué no incluye	4
6. Desarrollo	4
6.1 Análisis del problema logístico	4
6.3 Justificación del algoritmo heurístico.....	6
6.4 Implementación computacional	9
6.5 Evaluación del rendimiento de la solución	16
8. Conclusiones	21

2. Introducción

La creciente variabilidad climática y limitaciones de las técnicas agrícolas convencionales presentan retos significativos cuando de grandes extensiones de terreno se trata. El cambio climático, sequías prolongadas y temperaturas extremas nos obligan a siempre estar atentos a los cultivos y su estado de salud.[1]

En la agricultura de riego hay pérdidas de agua del orden del 50%, esto dado a que se usan volúmenes excesivos de agua y a la existencia de problemas para la operación y mantenimiento de la infraestructura de riego. De forma que el sobreuso de los recursos hídricos de México es de aproximadamente un 19.5%, o en zonas áridas a semiáridas puede haber un sobreuso de entre el 40% al 100%. [2]

Con el uso de técnicas agrícolas modernas se busca prevenir la pérdida de cultivos dada por sequías, enfermedades, peste, entre otras causas. La agricultura de precisión según Arely y Llano[3] se puede definir como “un conjunto de procedimientos y procesos que buscan optimizar espacial y temporalmente el ciclo de vida de diferentes cultivos a través de tecnologías, elementos y estudios de manera amigable con el medio ambiente” (pág. 106).

Kriegler[4] propuso una simple transformación de bandas: la radiación de la banda infrarrojo cercano (NIR) menos la radiación de la banda roja dividida entre el infrarrojo cercano más la radiación roja resulta una nueva imagen simplificada llamada Índice de Vegetación de Diferencia Normalizada o NDVI por sus siglas en inglés *Normalized Difference Vegetation Index*. Esta fue creada para tener una forma rápida de detectar vegetación en estrés hídrico.

Una de las herramientas que resultan útiles para la agricultura de precisión es QGIS, dado que es un sistema de información geográfico que nos ayuda en el procesamiento, análisis, proyecciones, modelamiento, toma de decisiones y más de los datos obtenidos del territorio y elementos que la componen, esto ya que la información obtenida se puede integrar, almacenar, editar, analizar, compartir y mostrar geográficamente referenciada en la interfaz del software.[5]

Todo esto, además de tener proveedores de algoritmos como GDAL (*Geospatial Data Abstraction Library*) la cual es una biblioteca traductora de formatos de datos geoespaciales ráster y vectoriales[6], nos abre un gran campo de oportunidades para el desarrollo de programas de agricultura de precisión.

Es por ello por lo que se propuso la creación de plug-ins en QGIS para el análisis de campos cultivos con el propósito de identificar plantas con estrés hídrico y en base a eso generar un plan de vuelo para un dron que visite dichas plantas que requieren atención. Ya sea para darles tratamiento o solo tomar fotos de estas para ver el estado en el que se encuentran.

3. Objetivos

Programa desarrollado para la detección de estrés hídrico en cultivos en base al índice NDVI calculado de las imágenes multiespectrales en las bandas RED y NIR (infrarrojo cercano) usando PyQGIS, así como la generación de un plan de vuelo para drones con el fin de visitar cada una de las plantas que estén en estrés hídrico.

3.1 Objetivo general

Identificar plantas con estrés hídrico basado en el índice NDVI con el fin de generar un plan de vuelo para un dron que ayude a solventar la situación de las plantas en dicha condición.

3.2 Objetivos específicos

- Identificar plantas con estrés hídrico en una parcela de cultivo mediante imágenes aéreas de drones.
- Conseguir las coordenadas de plantas con estrés hídrico.
- Trazar un plan de vuelo que visite las plantas con estrés hídrico en base al NDVI.

4. Alcance del proyecto

4.1 Qué incluye

- Integración de los scripts desarrollados en Python dentro del entorno QGIS mediante herramientas personalizadas.
- Procesamiento de imágenes multiespectrales correspondientes a las bandas NIR y RED para el cálculo del Índice de Vegetación de Diferencia Normalizada (NDVI).
- Generación automática de máscaras binarias que permitan identificar regiones con valores NDVI dentro de un rango determinado asociado a posibles condiciones de estrés hídrico.

- Conversión de máscaras ráster a capas vectoriales para el análisis de polígonos detectados.
- Filtrado de regiones según rangos mínimos y máximos de área configurables por el usuario.
- Generación automática de centroides para cada región de interés detectada.
- Ordenamiento de los centroides mediante un algoritmo de recorrido orientado a la generación de rutas para drones.
- Exportación de coordenadas y waypoints en formato CSV para su utilización posterior en sistemas de planificación de vuelo.
- Visualización de capas ráster, vectoriales, centroides y rutas dentro de QGIS.
- Compatibilidad con imágenes TIFF georreferenciadas.
- Incorporación de una herramienta adicional de georreferenciación para imágenes TIFF sin información espacial.
- Parametrización de los rangos de área mínima y máxima utilizados durante el filtrado de regiones de interés, permitiendo adaptar el algoritmo a diferentes escalas de cultivo.
- Generación automática de archivos temporales requeridos para el procesamiento sin necesidad de intervención manual del usuario.

4.2 Qué no incluye

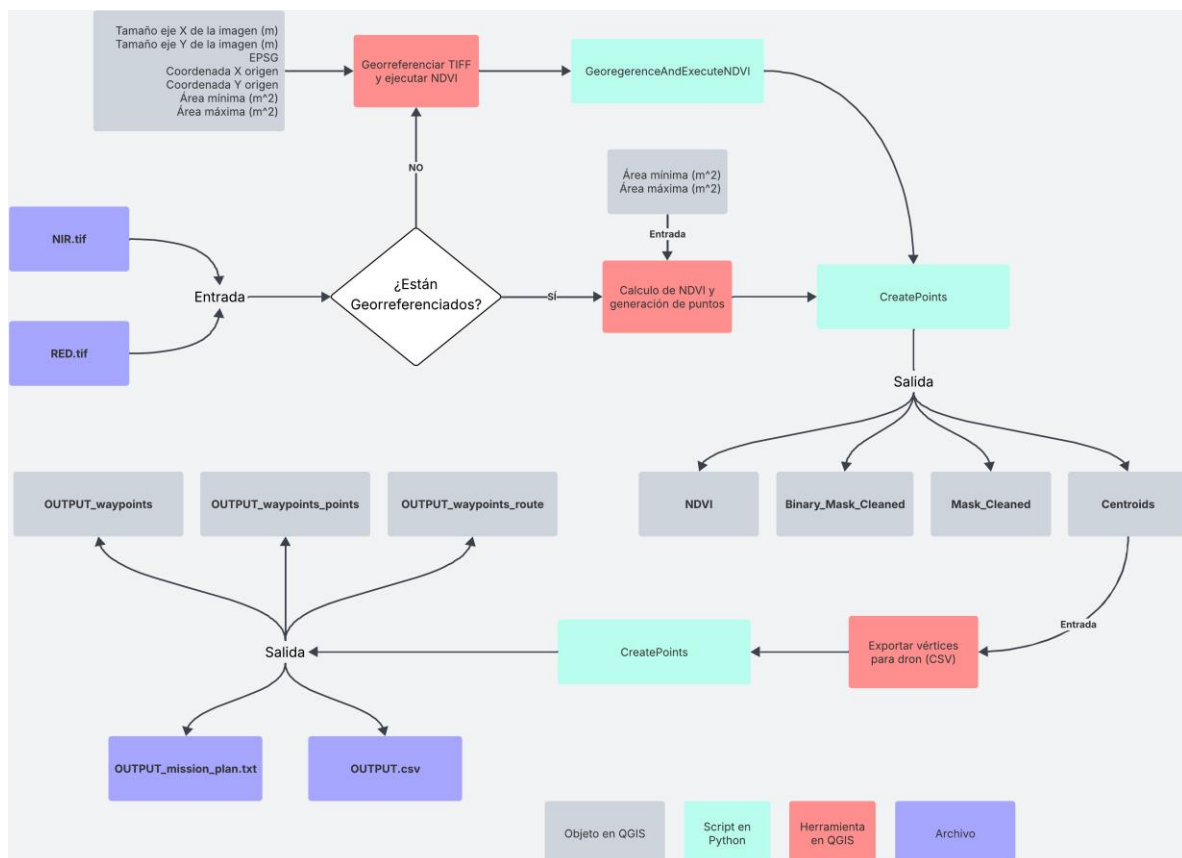
- Captura de imágenes mediante drones.
- Control en tiempo real de drones.
- Planeación automática de vuelos considerando obstáculos, restricciones aéreas o condiciones meteorológicas.
- Conversión automática de rutas a formatos propietarios de fabricantes específicos de drones.
- Identificación específica de enfermedades vegetales.
- Diagnóstico agronómico definitivo sobre el estado fisiológico de las plantas.
- Entrenamiento o desarrollo de modelos de inteligencia artificial para clasificación de cultivos.
- Corrección radiométrica de imágenes multiespectrales.
- Ortorrectificación o generación de mosaicos fotogramétricos de imágenes aéreas.
- Validación de la precisión geográfica de los datos generados.
- Desarrollo de una interfaz gráfica independiente de QGIS.

- Compatibilidad garantizada con versiones de QGIS distintas a la versión utilizada durante el desarrollo y pruebas del proyecto.
- Garantía de que los parámetros de área, escala o georreferenciación utilizados sean representativos de cualquier cultivo o escenario real.

5. Descripción del sistema

El sistema desarrollado tiene como objetivo identificar zonas de posible estrés hídrico dentro de un cultivo a partir de imágenes multispectrales capturadas por un dron y, posteriormente, generar una ruta optimizada que pueda ser recorrida por otro dron para realizar inspecciones o tratamientos localizados.

Las imágenes que se usan para el procesamiento fueron sacadas de un dataset de imágenes aéreas tomadas por un dispositivo aéreo no tripulado usando cámaras multispectrales (Verde, Rojo, Infrarrojo cercano) a tomates Cherry.[7]



El flujo de trabajo comienza con dos imágenes TIFF monobanda correspondientes a las bandas espectrales NIR (Near Infrared) y RED. Ambas imágenes representan la misma zona de cultivo capturada desde una plataforma aérea, pero registrando diferentes rangos del espectro electromagnético.

Etapa 1: Validación de georreferenciación

Antes de ejecutar el análisis NDVI se verifica si las imágenes poseen información geográfica válida. Si las imágenes ya se encuentran georreferenciadas, el sistema continúa directamente con la ejecución del algoritmo de cálculo NDVI. En caso de no estar georreferenciadas, se utiliza la herramienta **“Georreferenciar TIFF y ejecutar NDVI”**, la cual genera archivos GeoTIFF temporales que contienen la información espacial necesaria para que QGIS pueda realizar cálculos de áreas y coordenadas reales. Una vez completada la georreferenciación, el sistema ejecuta automáticamente el algoritmo principal de procesamiento.

Etapa 2: Cálculo del NDVI y máscara binaria y centroides

La herramienta **“Cálculo de NDVI y generación de puntos”** recibe como entrada las imágenes NIR y RED, con las que calcula el NDVI, que permite distinguir regiones potencialmente afectadas por estrés hídrico mediante el análisis de sus valores espectrales. El resultado de esta etapa es una capa ráster denominada: NDVI

Con el NDVI, se aplica un filtro por rango de valores. Los píxeles cuyos valores cumplen la condición establecida son marcados como regiones de interés, mientras que el resto son descartados. Como resultado se genera una máscara binaria donde los valores válidos representan posibles zonas con estrés hídrico. Posteriormente la máscara es limpiada y transformada a formato vectorial para permitir el cálculo de áreas y centroides.

Los polígonos obtenidos a partir de la máscara binaria son evaluados según su tamaño en m^2 , el usuario debe establecer un área mínima y un área máxima expresadas en m^2 desde los parámetros de la herramienta en QGIS. Aquellas regiones cuya superficie se encuentra fuera del rango especificado son descartadas automáticamente, esto para eliminar ruido generado por objetos demasiado pequeños o grandes que no representan plantas o zonas de interés para la inspección.

Después del filtrado de áreas se calcula el centro geométrico de cada polígono válido, cada uno de estos puntos representa una ubicación candidata para ser visitada por un dron durante una misión de inspección. El resultado de esta etapa es la capa “Centroids”.

Etapa 6: Generación y exportación de waypoints y plan de vuelo

La capa de centroides se utiliza como entrada de la herramienta “**Exportar vértices para dron (CSV)**”, que procesa todas las coordenadas generadas y calcula un orden de recorrido para los puntos detectados. Durante este proceso se generan diferentes capas auxiliares dentro de QGIS que permiten visualizar gráficamente el orden de visita y la trayectoria calculada.

Finalmente, el sistema exporta los resultados a archivos externos “**OUTPUT.csv**” y “**OUTPUT_mission_plan.txt**”, que pueden ser utilizados por aplicaciones de planificación de vuelo o por software especializado para drones. El archivo CSV contiene las coordenadas ordenadas de los puntos detectados en un formato versátil para drones, mientras que el archivo txt almacena la información en una forma comprensible de caracteres de texto.

7. Estructura y funcionamiento de los componentes

El Proyecto se compone de dos scripts escritos en Python hechos para funcionar con el software de QGIS

ScriptWaypointsExtraction.py

```
def calculate_NDVI(self, nir_layer: QgsRasterLayer, red_layer: QgsRasterLayer, output_path: str, source_crs):
    # se cargan las imagenes a la calculadora raster
    nir_entry = QgsRasterCalculatorEntry()
    nir_entry.raster = nir_layer
    nir_entry.ref = 'nir@1'
    nir_entry.bandNumber = 1

    red_entry = QgsRasterCalculatorEntry()
    red_entry.raster = red_layer
    red_entry.ref = 'red@1'
    red_entry.bandNumber = 1

    entries = [nir_entry, red_entry]

    # se guardan las propiedades de una de las imagenes
    extent = nir_layer.extent()
    width = nir_layer.width()
    height = nir_layer.height()
    #expresion del NDVI
    expression = '(nir@1 - red@1) / (nir@1 + red@1 + 1e-10)'

    calc = QgsRasterCalculator(expression, output_path, 'GTiff', extent, width, height, entries)
    result = calc.processCalculation()

    if result != 0:
        raise QgsProcessingException(f'Error al calcular NDVI, código: {result}')

    ndvi_layer = QgsRasterLayer(output_path, 'NDVI')
    if ndvi_layer.isValid():
        ndvi_layer.setCrs(source_crs)
        QgsProject.instance().addMapLayer(ndvi_layer)
```

Método usado solo para calcular el índice NDVI del terreno. La fórmula usada es

$$NDVI = \frac{(NIR - RED)}{(NIR + RED)}$$

El calculo funciona tomando los realizando la operación por cada píxel, después de todo, las imágenes son matrices de números.

```
# Calculo de la máscara binaria segun la condicion dada
def calculate_mask(self, ndvi_layer: QgsRasterLayer, output_path: str, source_crs):
    bMask_entry = QgsRasterCalculatorEntry()
    bMask_entry.raster = ndvi_layer
    bMask_entry.ref = 'ndvi@1'
    bMask_entry.bandNumber = 1

    entries = [bMask_entry]

    extent = ndvi_layer.extent()
    width = ndvi_layer.width()
    height = ndvi_layer.height()

    expression = '(ndvi@1 < 0.333) AND (0.12 < ndvi@1)'

    calc = QgsRasterCalculator(expression, output_path, 'GTiff', extent, width, height, entries)
    result = calc.processCalculation()

    if result != 0:
        raise QgsProcessingException(f'Error al calcular máscara binaria, código: {result}')

    return output_path
```

Una vez más se usa la calculadora, pero ahora para crear una máscara binaria con las zonas que cumplen la condición (0's y 1's). Se explicará el por que se usa dicha expresión más adelante.

En **mask_to_vector** este proceso, se recibe la máscara binaria calculada anteriormente, se convierte en un vector usando la herramienta de gdal. Según la documentación oficial disponible en <https://docs.qgis.org>: “Crea polígonos vectoriales para todas las regiones conectadas de píxeles en el ráster que comparten un valor de píxel común. Cada polígono se crea con un atributo que indica el valor de píxel de ese polígono”.

Luego se calcula el área de cada uno de los vectores individuales. Según su descripción disponible en la aplicación de QGIS (usando el comando: `processing.algorithmHelp("native:fieldcalculator")`):

```

# Convertir máscara binaria a vectorial y limpiar polígonos según área
def mask_to_vector(self, mask_path: str, ndvi_layer: QgsRasterLayer, source_crs):

    mask_vector = os.path.splitext(mask_path)[0] + "_vector.gpkg"
    processing.run("gdal:polygonize", {
        'INPUT': mask_path,
        'BAND': 1,
        'FIELD': 'value',
        'EIGHT_CONNECTEDNESS': False,
        'OUTPUT': mask_vector
    })

    mask_with_area = os.path.splitext(mask_path)[0] + "_area.gpkg"
    processing.run("native:fieldcalculator", {
        'INPUT': mask_vector,
        'FIELD_NAME': 'area_m2',
        'FIELD_TYPE': 0, # decimal
        'FIELD_LENGTH': 20,
        'FIELD_PRECISION': 3,
        'FORMULA': '$area',
        'OUTPUT': mask_with_area
    })

    min_area = 50
    max_area = 500

    cleaned_vector = os.path.splitext(mask_path)[0] + "_cleaned.gpkg"
    processing.run("native:extractbyexpression", {
        'INPUT': mask_with_area,
        'EXPRESSION': f'"value" = 1 AND "area_m2" >= {min_area} AND "area_m2" <= {max_area}',
        'OUTPUT': cleaned_vector
    })
    calculated_vector = QgsVectorLayer(cleaned_vector, 'Mask_Cleaned', 'ogr')
    if calculated_vector.isValid():
        calculated_vector.setCrs(source_crs)

    extent = ndvi_layer.extent()
    cleaned_raster = os.path.splitext(mask_path)[0] + "_cleaned.tif"
    processing.run("gdal:rasterize", {
        'INPUT': cleaned_vector,
        'FIELD': 'value',
        'UNITS': 1,
        'WIDTH': ndvi_layer.rasterUnitsPerPixelX(),
        'HEIGHT': ndvi_layer.rasterUnitsPerPixelY(),
        'EXTENT': extent,
        'NODATA': 0,
        'DATA_TYPE': 5,
        'OUTPUT': cleaned_raster
    })

```

Según su descripción “Este algoritmo calcula una nueva capa vectorial con los mismos objetos de la capa de entrada, pero sobrescribiendo un atributo existente o agregando un atributo adicional. Los valores de este campo son calculados para cada objeto usando una expresión, basada en las propiedades y atributos del objeto.”

Posteriormente, se filtra la capa conseguida dadas dos áreas. Según su descripción: “Este algoritmo crea una nueva capa vectorial que solo contiene objetos

coincidentes de una capa de entrada. Los criterios para agregar objetos a la capa resultante se basan en una expresión QGIS.”

Por último, se recibe la capa de vectores con las áreas filtradas y se transforma nuevamente a un ráster. Según la documentación oficial: “Convierte geometrías vectoriales (puntos, líneas y polígonos) en una imagen ráster.”

Una vez obtenida la capa de vectores con las áreas ya filtradas, se procede a calcular los centroides de las zonas resultantes.

```
# Calculo de los centroides usando las herramientas nativas
def calculate_centroids(self, vector_path: str, source_crs):
    centroids_path = os.path.splitext(vector_path)[0] + "_centroids.gpkg"
    processing.run("native:centroids", {
        'INPUT': vector_path,
        'OUTPUT': centroids_path
    })
    centroids_layer = QgsVectorLayer(centroids_path, 'Centroids', 'ogr')
    if centroids_layer.isValid():
        centroids_layer.setCrs(source_crs)
        QgsProject.instance().addMapLayer(centroids_layer)
    else:
        raise QgsProcessingException("No se pudo cargar el layer de centroides.")
    return centroids_path
```

Según su descripción: *“Este algoritmo crea una capa de puntos nueva, con puntos que representan el centroe de las geometrías de la capa de entrada”.*

En el caso de ScriptPointsGenerator.py, la lógica del método principal es que una vez obtenidos los centroides (resultado del script anterior), se extra el primer punto de la lista y se inicia un bucle en el que se recorre todos los puntos para ver cual punto está más al extremo. Una vez se tiene, se crean dos listas nuevas.

- **Optmized_route:** aquí se guardaran los puntos en el orden de visita. En la posición 0 está el punto más al extremo.
- En **unvisited_points:** todos los puntos que no coinciden con el punto más al extremo.

Posteriormente, se inicia un bucle while que continua mientras haya puntos en unvisited_points. Dentro de este while se recorre la lista de unvisited_pointes, esto con el propósito de medir la distancia usando el teorema de Pitágoras.

```

def optimize_route(self, points):
    """Optimiza la ruta de puntos usando algoritmo greedy del vecino más cercano"""
    if not points:
        return []

    # Encontrar punto inicial (lo más cercano a una esquina)
    edge_point = points[0]
    for point in points:
        if abs(point[0]) < abs(edge_point[0]) or abs(point[1]) < abs(edge_point[1]):
            edge_point = point

    optimized_route = [edge_point]
    unvisited_points = [p for p in points if p != edge_point]

    while unvisited_points:
        min_distance = float('inf')
        next_point = None

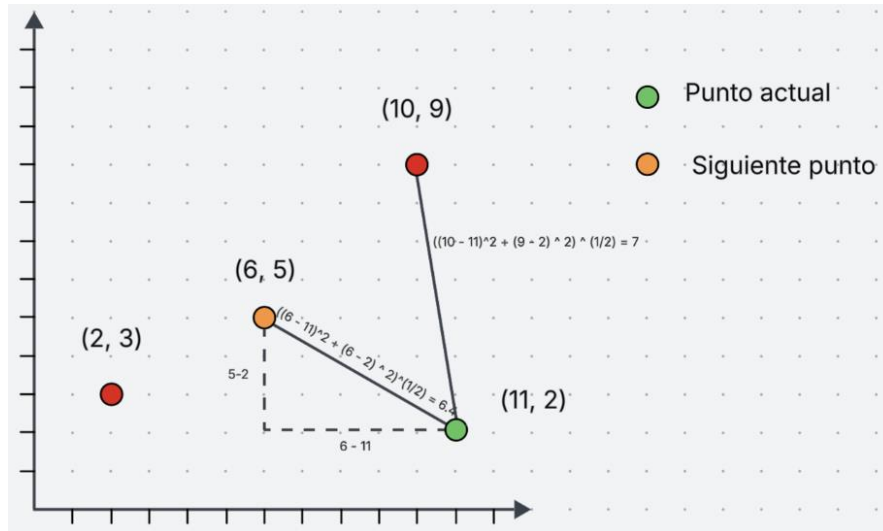
        for point in unvisited_points:
            distance = sqrt(
                pow(point[0] - edge_point[0], 2) +
                pow(point[1] - edge_point[1], 2)
            )

            if distance < min_distance:
                min_distance = distance
                next_point = point

        if next_point:
            optimized_route.append(next_point)
            unvisited_points.remove(next_point)
            edge_point = next_point
        else:
            break

    return optimized_route

```

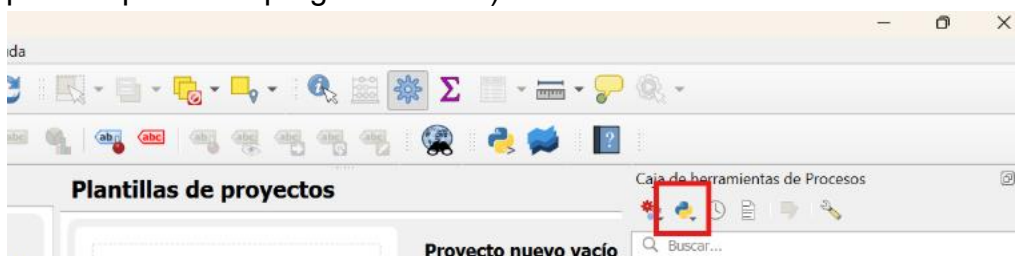


8. Requisitos del sistema

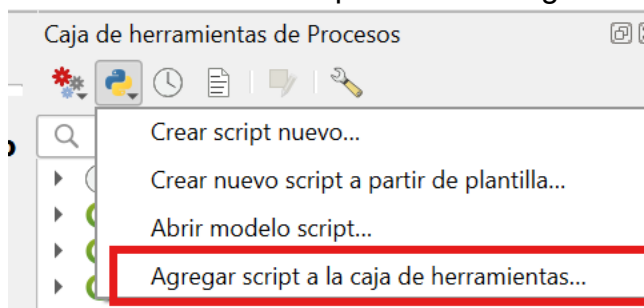
- QGIS versión 3.40.12 (la usada para las pruebas y desarrollo) que se puede descargar desde <https://download.osgeo.org/qgis/windows/> en el archivo QGIS-OSGeo4W-3.40.12-1.msi de 1.3 GiB del 2025-Oct-24 22:14
- Python 3.12 o superior
- Las librerías requeridas vienen preinstaladas con QGIS, pues tiene una terminal de python con su propia máquina virtual.

9. Instrucciones de implementación

1. Una vez instalado QGIS dar clic izquierdo sobre el ícono de Python (en versiones recientes de QGIS antes debes de abrir la ventana en la opción “procesos” que está en la parte superior del programa GGIS)



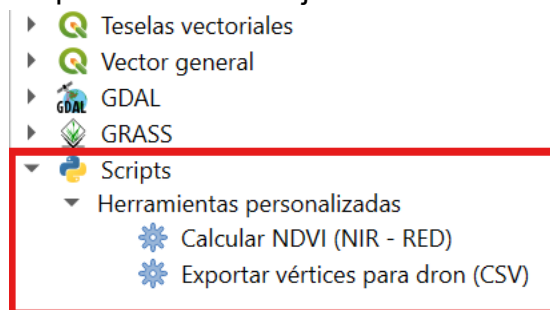
2. Se desplegará un menú. Damos clic izquierdo en la siguiente opción



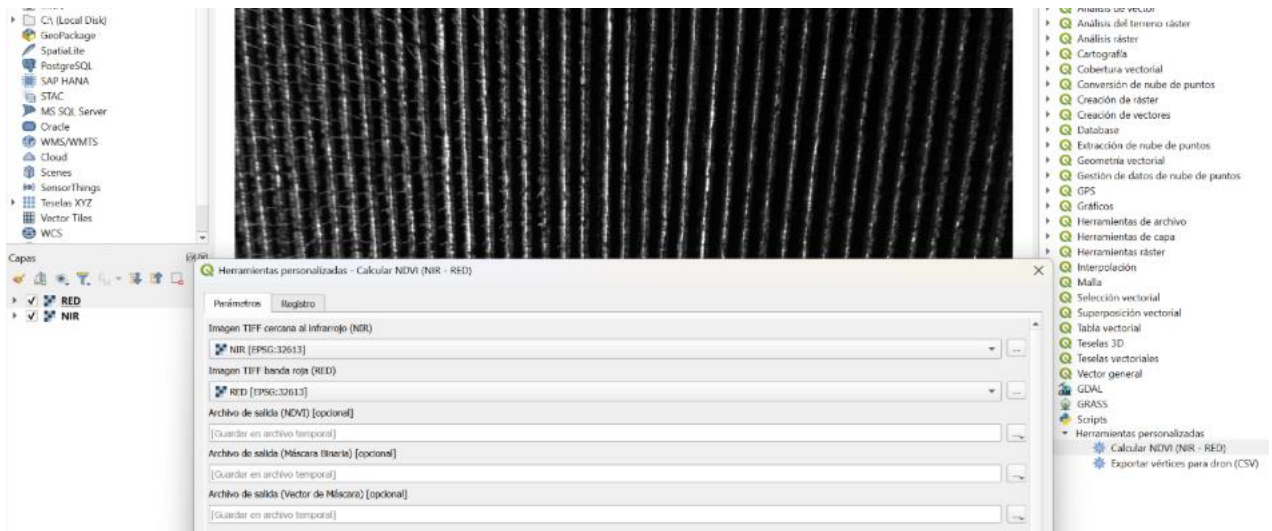
Se abrirá el explorador de archivos. Seleccionamos los dos Scripts.

1.- ScriptPointsGenerator.py 2.- ScriptWaypointsExtractionv2.py

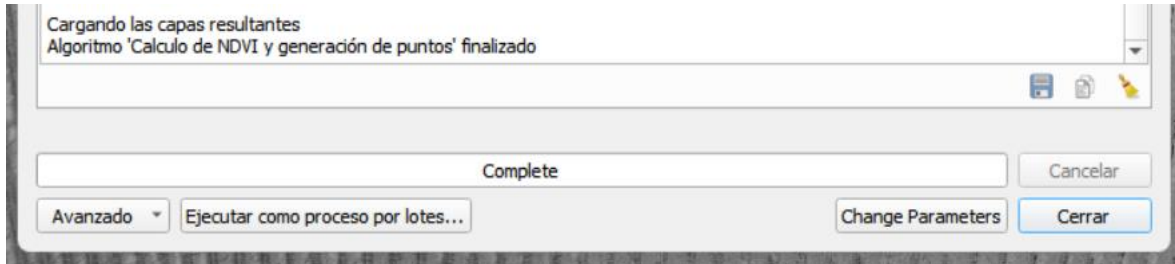
Ahora las herramientas aparecen en la “caja de herramientas”



3. Para usarlas damos doble clic izquierdo. Primero sobre la primera herramienta “Calcular NDVI (NIR – RED)”.

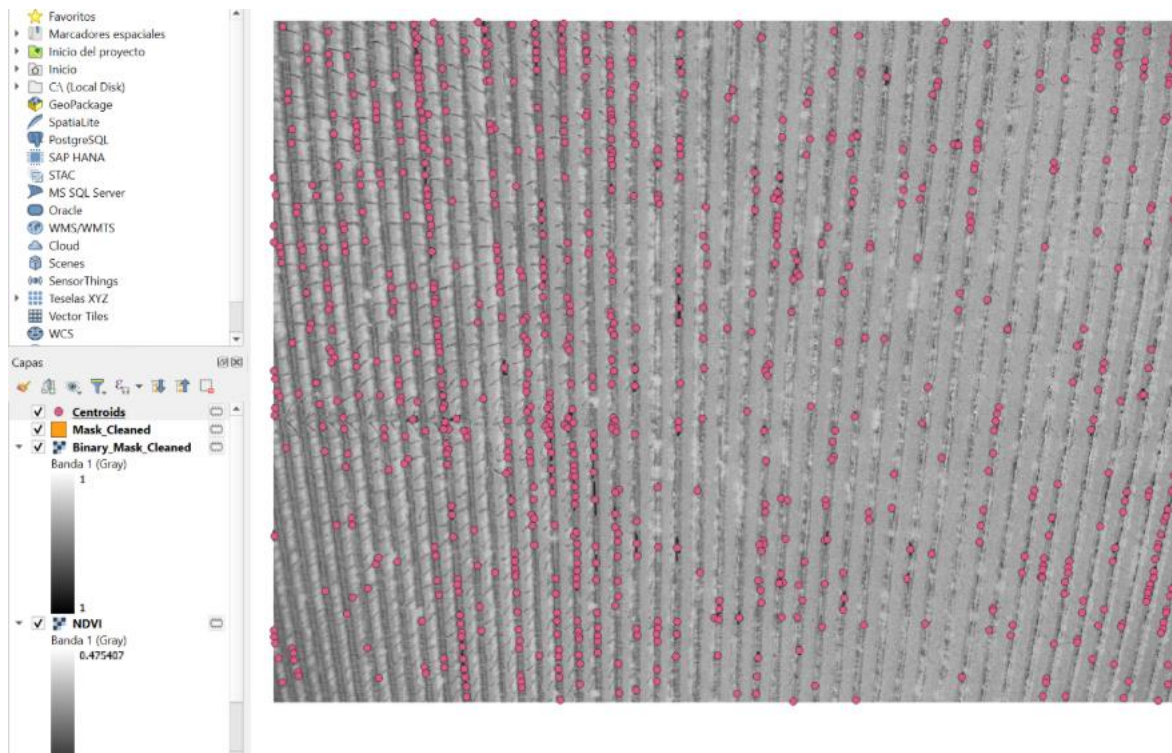


Y seleccionamos las imágenes monobanda(monocromas o unibanda) con las bandas NIR y RED previamente tomadas. Y dejamos lo de las salidas vacías y damos en “Ejecutar”.

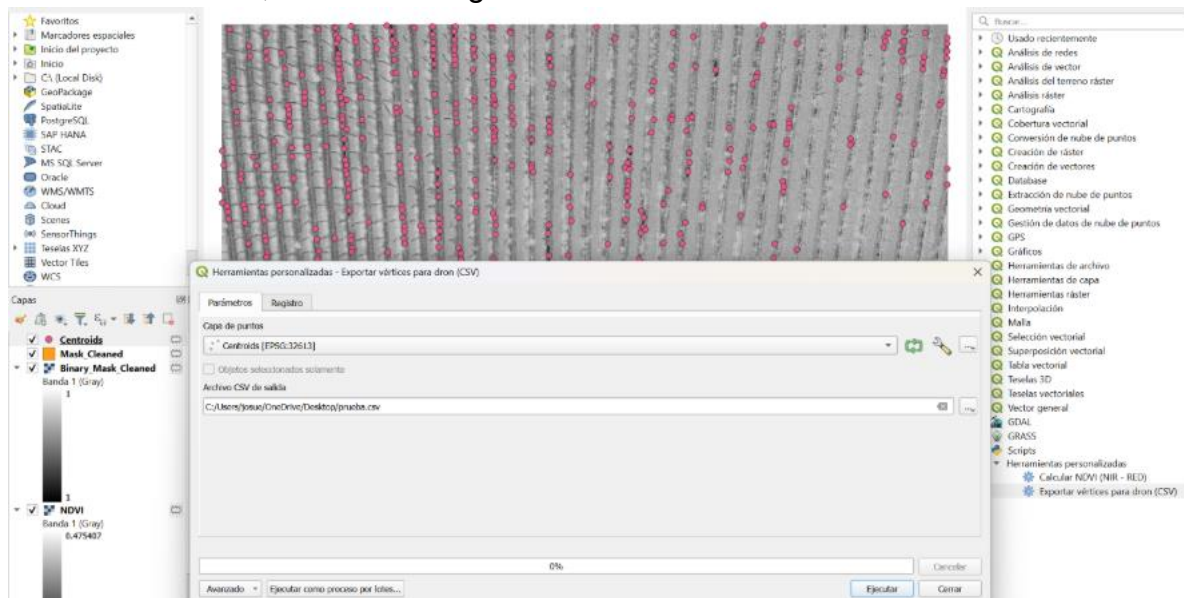


Mostrará el siguiente apartado, confirmando que el proceso ha sido completado sin inconvenientes.

El resultado debe ser similar al siguiente.



4. Posteriormente, usamos la segunda herramienta



A la cual le daremos como parámetro de entrada los centroides calculados y estableceremos una ruta de salida para los archivos. Damos "Ejecutar". La salida debe ser la siguiente:



Todos los puntos numerados en el orden de visita, además de la generación de los archivos en la ruta especificada

El CSV tendrá el siguiente formato

1	lon	lat	alt	orden
2	-109.45302759	-0.02254173	20	0
3	-109.45310022	-0.02284136	20	1
4	-109.45321883	-0.02337874	20	2
5	-109.45325689	-0.02350555	20	3
6	-109.45337051	-0.02389711	20	4
7	-109.4533878	-0.02415899	20	5
8	-109.45346192	-0.02429957	20	6
9	-109.453186	-0.02557569	20	7
10	-109.45319086	-0.0263575	20	8
11	-109.4535606	-0.02701158	20	9
12	-109.45494186	-0.02690744	20	10
13	-109.454764	-0.02621572	20	11
14	-109.45534236	-0.0255414	20	12
15	-109.45618186	-0.02534323	20	13
16	-109.45678823	-0.02470013	20	14
17	-109.45677355	-0.0245569	20	15
18	-109.45770573	-0.02475593	20	16
19	-109.45799607	-0.02612552	20	17
20	-109.45803916	-0.02628507	20	18
21	-109.45816648	-0.02683655	20	19
22	-109.45725055	-0.02681602	20	20

Y el plan de vuelo se debe ver

1	QGC	WPL	110									
2	0	1	0	16	0	0	0	0	-0.02254173	-109.45302759	13	1
3	1	0	3	16	0	0	0	0	-0.02284136	-109.45310022	45	1
4	2	0	3	16	0	0	0	0	-0.02337874	-109.45321883	45	1
5	3	0	3	16	0	0	0	0	-0.02350555	-109.45325689	45	1
6	4	0	3	16	0	0	0	0	-0.02389711	-109.45337051	45	1
7	5	0	3	16	0	0	0	0	-0.02415899	-109.4533878	45	1
8	6	0	3	16	0	0	0	0	-0.02429957	-109.45346192	45	1
9	7	0	3	16	0	0	0	0	-0.02557569	-109.453186	45	1
10	8	0	3	16	0	0	0	0	-0.0263575	-109.45319086	45	1
11	9	0	3	16	0	0	0	0	-0.02701158	-109.4535606	45	1
12	10	0	3	16	0	0	0	0	-0.02690744	-109.45494186	45	1
13	11	0	3	16	0	0	0	0	-0.02621572	-109.454764	45	1
14	12	0	3	16	0	0	0	0	-0.0255414	-109.45534236	45	1
15	13	0	3	16	0	0	0	0	-0.02534323	-109.45618186	45	1
16	14	0	3	16	0	0	0	0	-0.02470013	-109.45678823	45	1
17	15	0	3	16	0	0	0	0	-0.0245569	-109.45677355	45	1
18	16	0	3	16	0	0	0	0	-0.02475593	-109.45770573	45	1
19	17	0	3	16	0	0	0	0	-0.02612552	-109.45799607	45	1
20	18	0	3	16	0	0	0	0	-0.02628507	-109.45803916	45	1
21	19	0	3	16	0	0	0	0	-0.02683655	-109.45816648	45	1
22	20	0	3	16	0	0	0	0	-0.02681602	-109.45725055	45	1
23	21	0	3	16	0	0	0	0	-0.02403382	-109.45594169	45	1
24	22	0	3	16	0	0	0	0	-0.02337601	-109.45573285	45	1
25	23	0	3	16	0	0	0	0	-0.02351677	-109.45659461	45	1
26	24	0	3	16	0	0	0	0	-0.02303006	-109.45654074	45	1
27	25	0	3	16	0	0	0	0	-0.0227517	-109.45731205	45	1
28	26	0	3	16	0	0	0	0	-0.02228648	-109.45727925	45	1
29	27	0	3	16	0	0	0	0	-0.02191486	-109.45714375	45	1
30	28	0	3	16	0	0	0	0	-0.0217154	-109.45714072	45	1
31	29	0	3	16	0	0	0	0	-0.02144317	-109.45709269	45	1
32	30	0	3	16	0	0	0	0	-0.02134463	-109.45809544	45	1
33	31	0	3	16	0	0	0	0	-0.0216349	-109.4581507	45	1
34	32	0	3	16	0	0	0	0	-0.02178566	-109.45814897	45	1
35	33	0	3	16	0	0	0	0	-0.02209833	-109.45817991	45	1
36	34	0	3	16	0	0	0	0	-0.02208681	-109.45824724	45	1
37	35	0	3	16	0	0	0	0	-0.02101712	-109.45809224	45	1

10. Conclusiones

La implementación y análisis del sistema permitió comprender el flujo completo de procesamiento utilizado para transformar imágenes multiespectrales de un cultivo en una ruta de navegación para drones. A partir de las bandas NIR y RED se calcula el índice NDVI, posteriormente se identifican regiones de interés mediante máscaras binarias, se generan centroides representativos y finalmente se obtiene una secuencia ordenada de coordenadas exportable a formatos compatibles con sistemas de planificación de vuelo. De esta forma, el sistema automatiza tareas que de otra manera requerirían un análisis manual considerable, facilitando la detección y localización de posibles zonas con estrés hídrico.

Durante el proceso de implementación también se identificaron limitaciones importantes en los datos de ejemplo proporcionados por el repositorio original, particularmente la ausencia de información de georreferenciación en las imágenes TIFF utilizadas para las pruebas. Esta situación provocaba resultados inconsistentes en los cálculos de área y en la generación de centroides, por lo que

fue necesario desarrollar una herramienta complementaria de georreferenciación y parametrizar ciertos valores del algoritmo para adaptarlo a diferentes escalas de trabajo. Como resultado, el sistema quedó más flexible y robusto, permitiendo procesar imágenes con o sin georreferenciación previa y facilitando futuras adaptaciones a distintos escenarios agrícolas o conjuntos de datos.

11. Diario de implementación

1- Importación del proyecto

Se descomprimió el proyecto en archivo .zip. Se importó la carpeta " Repositorios" del proyecto a Visual Studio Code con la siguiente estructura

Repositorios

- | AUTOMATIC FLIGHT PLAN.docx
- | Diagrama de Automatic-FlightPlan.jpg
- | ScriptWaypointsExtraction.py
- |
 - Automatic-FlightPlan-main
 - points.csv
 - README.md
 - ScriptWaypointsExtraction.py
 - waypoints.csv
 - Automatic-UAV_waypoints-main
 - Documentos.txt
 - README.md
 - ScriptPointsGenerator.py
 - ScriptWaypointsExtractionv2.py
 - imagenes
 - ndviprueba.tif
 - NIR.tif
 - RED.tif
- | Nueva_Documentacion
 - README.md

2- Instalación de QGIS

La versión de QGIS que se indica como ideal en la documentación del proyecto es 3.40.12, pero en el repositorio de la página de descarga de QGIS recomendada es <https://qgis.org/downloads-list/#windows> donde no aparece ninguna versión 3.40.x para Windows. La versión más cercana recomendada es 3.42.3, específicamente el archivo QGIS-OSGeo4W-3.42.3-1.msi (evitar confundir la versiones QGIS-OSGeo4W con las versiones QGISQT6-OSGeo4W, que usan distintas importaciones)

Por otro lado, se logra obtener la versión 3.40.12 mediante <https://download.osgeo.org/qgis/windows/> en el archivo QGIS-OSGeo4W-3.40.12-1.msi de 1.3 GiB del 2025-Oct-24 22:14, esta versión es la que en esta implementación se descarga e instala.

3- Configuración de QGIS

Al abrir QGIS se confirma la versión en la **barra superior** -> **"Ayuda"** -> **"Acerca de"**, confirmando que la Versión de QGIS sea 3.40.12-Bratislava.

De nuevo en la **barra superior** -> **"Procesos"** -> **"Caja de Herramientas"**, aparece un panel a la derecha "Caja de herramientas de procesos", en la parte superior del panel se selecciona el logo de **Python** -> **"Agregar script a la caja de herramientas"**, se abrirá el explorador de archivos, en la carpeta **"Automatic-UAV_waypoints-main"** del repositorio se seleccionan los archivos **"ScriptPointsGenerator.py"** y **"ScriptWaypointsExtractionv2.py"**.

En el panel lateral **"Caja de herramientas de procesos"** hasta abajo -> **"Scripts"** (logo de python) -> **"Herramientas personalizadas"** deben aparecer dos herramientas **"Calculo de NDVI y generación de puntos"** y **"Exportar vértices para dron (CSV)"**

4- Primera Ejecución

En la **barra superior** -> **"Proyecto"** -> **"Nuevo"** se abrirá un proyecto en blanco. En el panel lateral derecho **"Caja de herramientas de procesos"** -> **"Scripts"** -> **"Herramientas personalizadas"** -> **"Calculo de NDVI y generación de puntos"**, se ingresan como parámetros las imágenes NIR.tif y RED.tif de la carpeta **"Automatic-UAV_waypoints-main\imagenes"** y se ejecuta.

Ejecución completada en 29.35 segundos

Results:

OUTPUT:

C:\Users\usuario\AppData\Local\Temp\processing_TCTWRZ\91281a7c8ee840638653492deaafa405\OUTPUT.tif

OUTPUT_MASK:

C:\Users\usuario\AppData\Local\Temp\processing_TCTWRZ\cc49a24719194e54b7c5da16373ea715\OUTPUT_MASK_cleaned.tif

OUTPUT_VECTOR:

C:\Users\usuario\AppData\Local\Temp\processing_TCTWRZ\cc49a24719194e54b7c5da16373ea715\OUTPUT_MASK_cleaned.gpkg

El algoritmo se ejecuta exitosamente pero el resultado no es el esperado, todo se filtra por errores de georreferenciación, El resto del algoritmo funciona correctamente, pero hay un problema que está en los archivos dados sin una georreferenciación, y esto se comprueba de dos formas:

1- los archivos NIR.tif y RED.tif incluidos en el repositorio, no contienen un Sistema de Referencia de Coordenadas (CRS) definido. En el panel izquierdo de **"capas"** en QGIS, al hacer clic derecho en **cualquier mapa** (NDVI, NIR, RED...) -> **"propiedades"** -> **"Información"**, se reporta CRS desconocido, origen (0,0) y tamaño de píxel (1,-1). Esto impide asociar los píxeles con coordenadas reales y pone en duda la validez de los cálculos de área utilizados posteriormente para filtrar polígonos y generar centroides.

2- El algoritmo genera archivos temporales con áreas que posteriormente se filtran, estos archivos no son finales, pero pueden ser encontrados en el historial de procesos después de la ejecución del algoritmo. En este caso de ejecución, el archivo temporal se encuentra en

C:\Users\usuario\AppData\Local\Temp\processing_TCTWRZ\cc49a24719194e54b7c5da16373ea715\OUTPUT_MASK_area.gpkg

Al arrastrar este archivo al panel de capas de QGIS, hacer clic derecho y mostrar su tabla de atributos se pueden notar 129178 registros de áreas calculadas con tamaños de números de 11 dígitos en metros cuadrados; el algoritmo a esto le aplica un filtro de tamaño de entre 50 y 500 metros cuadrados. Al estar desproporcionado, ningún área generada por el algoritmo pasa el filtro, por lo que Mask_Cleaned (el archivo de áreas resultante) queda sin registros, por lo tanto no quedan áreas a las cuales generar un centroide y el algoritmo no continúa, la capa tanto de áreas como de centroides quedan vacías.

5- Nuevo algoritmo de georreferenciación

Ante este problema se crea un nuevo algoritmo de georreferenciación para los archivos .tif ScriptTifGeoreferenciator.py, el archivo se guarda junto a los demás scripts (en la carpeta "Automatic-UAV_waypoints-main") y se importa a QGIS de la misma forma. Aparece en la Caja de Herramientas Personalizadas de QGIS como "Georreferenciar TIFF y ejecutar NDVI", sus nuevos parámetros los tamaños del eje X y Y de la imagen .tif en metros, cuyos valores predeterminados son 200.0,150.0 y se recomienda dejarlos así en el ejemplo. También el formato EPSG (formato de representación de longitudes en las imágenes) cuyo valor predeterminado es 32613. Y los últimos parámetros que se recomiendan no mover son las coordenadas X,Y de origen de la imagen. Los siguientes parámetros son los mismos que en el algoritmo NDVI original, pues al georreferenciar la imagen .tif el algoritmo ya funciona correctamente.

6- Cambios de parámetros en el algoritmo NDVI

Se instancia el nuevo algoritmo Georreferenciador y funciona correctamente, pero se presenta un nuevo problema: genera 11 centroides a diferencia de los más de 800 (aproximados) que genera la ejecución del algoritmo según la documentación original. La teoría es que las variables del algoritmo del filtro por el tamaño de las áreas de estrés, al ser establecidos al tanteo, generan inconsistencias ante las nuevas proporciones de longitud de imagen establecidas como parámetro en el nuevo algoritmo. Es por esto que se modificó el script del cálculo NDVI (el script original) para convertir en parámetros de QGIS las variables de mínimo y máximo de metros cuadrados del filtro de áreas, evitando así tener que modificar, re importar y compilar el script cada vez que se quiera hacer un cambio en los valores del filtro.

Con estos cambios se resuelven varios problemas que tenía el repositorio dado, el algoritmo ahora genera áreas y calcula centroides de una forma más versátil, aceptando archivos .tif con o sin georreferenciación. Este nuevo algoritmo (ahora con parámetros del filtro de áreas) se aplica en las imágenes NIR y RED dados de ejemplo en el repositorio, que tanteando se llegó a la conclusión de que con valores de área mínima y máxima de 0.13 y 0.5 metros cuadrados (con el tamaño de imagen predeterminado de 200x150 metros), se obtiene un resultado similar al de la ejecución del algoritmo presentado en la documentación del proyecto: se obtienen 710 centroides, con una distribución similar a comparación de los 842 (aproximados, no se especifica el número real en la documentación) obtenidos con el algoritmo anterior, el cual tenía una configuración de filtro de áreas de entre 50 y 500 metros cuadrados, algo desproporcionado con la realidad.

7- Ejecutar el algoritmo de ordenamiento de los centroides

Después del cálculo de los centroides con una prueba de filtro de áreas de entre 0.1 y 0.5 metros cuadrados, en **"Caja de herramientas de procesos" -> "Scripts" -> "Herramientas personalizadas"** se ejecuta **"Exportar vértices para dron (CSV)"** se selecciona la capa de centroides generada por el anterior script, en este caso **"Centroids [EPSG:32613]"**. Después de ejecutarse el algoritmo deben aparecer nuevas capas en el panel izquierdo de **"Capas"**:

- "centroides_waypoints"
- "centroides_waypoints_points"
- "centroides_waypoints_route"

Pero aquí parece presentarse otro problema: la documentación muestra que el resultado en el mapa deben de ser números sobre cada centroide, pero el resultado del panel de mapa después de la ejecución a veces puede no mostrar nada de eso, esto es un problema meramente visual de QGIS, pues por otra parte, en el panel izquierdo de **"Capas"**, al hacer clic derecho en **"centroides_waypoints"** o en **"centroides_waypoints_points" -> "Abrir tabla de atributos"** se muestra una tabla con 1011 elementos (la misma cantidad de centroides) con atributos de longitud, latitud y orden, lo que indica que el archivo CSV se sigue generando de manera correcta. Otro factor que puede desencadenar a que no aparezcan los números en pantalla es que en el panel izquierdo de "Capas", al desactivar la casilla marcada de "OUTPUT_waypoints_points" desaparezcan los números (normal), pero al volver a activar la casilla no vuelvan a aparecer. Que no aparezcan los números en el panel principal no quiere decir que los puntos y sus coordenadas no existan o se hayan borrado, el algoritmo funciona correctamente si en el panel de "Capas" existen los archivos de:

- "centroides_waypoints"
- "centroides_waypoints_points"
- "centroides_waypoints_route"

Además, sus tablas de atributos muestran tantos registros como centroides generados en sus etapas anteriores.

n. Referencias

- [1] (Gómez, 2025) [View of Determinación del rendimiento del cultivo de maíz mediante el análisis de imágenes con Python](#) (Gómez, 2025)
- [2] (Bermúdez, et al., 2023) [Vista de Detección del nivel de estrés hídrico en plantas de lechuga romana a través de CNN](#)
<https://repository.uaeh.edu.mx/revistas/index.php/icbi/article/view/10943/1052>
(Bermúdez)
- [3] Arely y Llano (2015) [A commentary review on the use of normalized difference vegetation index \(NDVI\) in the era of popular remote sensing | Journal of Forestry Research](#) (Arely y Llano)
- [4] Kriegler et al. (1969) citado en Huang et al. (2020)
<https://www.scirp.org/journal/paperinformation?paperid=30705> k
- [5] ola [▷ QGIS Que Es y Para Que Sirve: todo lo que debes saber](#) ola
- [6] 24.2. Proveedor de algoritmos GDAL — documentación de QGIS
Documentation
https://docs.qgis.org/3.40/es/docs/user_manual/processing_algs/gdal/index.html a
- [7] a [Dryad | Data: Dataset of aerial photographs acquired with UAV using a multispectral \(Green, Red and Near-infrared\) camera for cherry tomato \(Solanum lycopersicum var. cerasiforme\) monitoring](#) a